

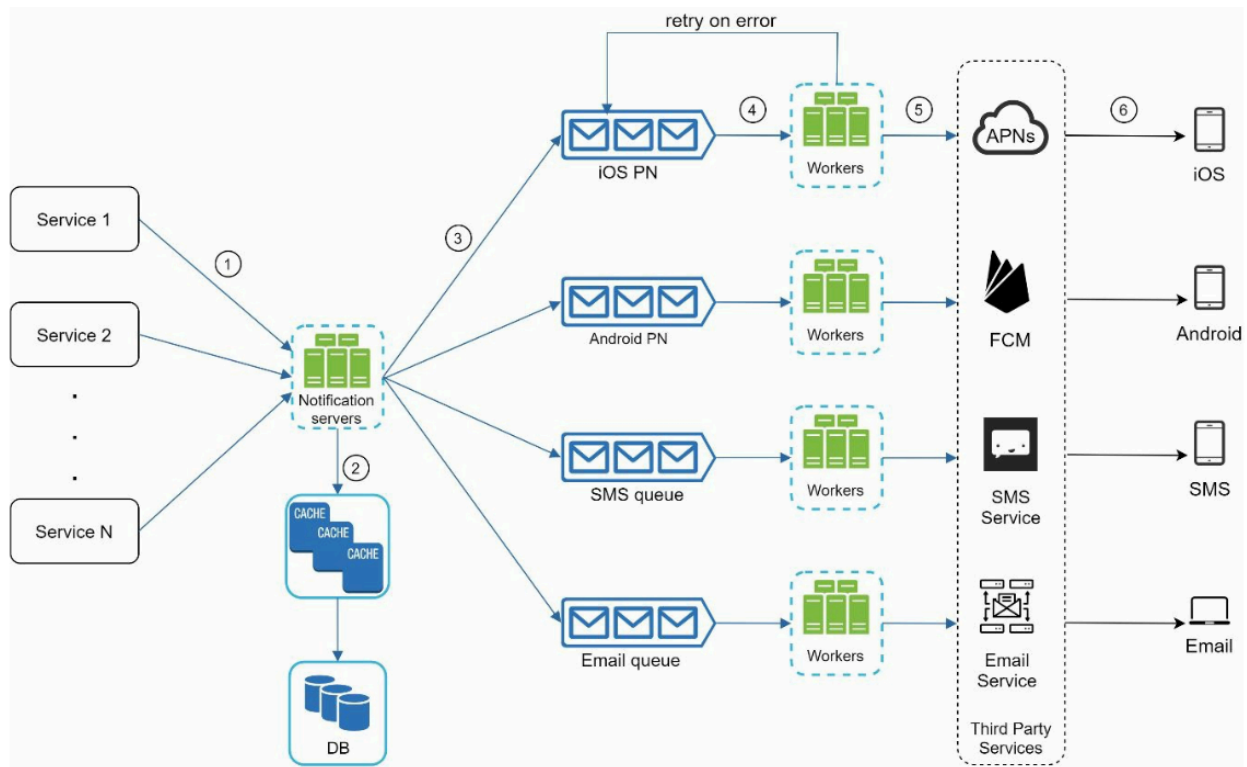
High-Level Design (Improved)

After identifying the limitations in the initial design, the notification system architecture is improved to make it:

- More scalable
- More reliable
- More fault tolerant
- Easier to maintain

The major improvements are:

- Move database and cache outside notification servers
- Add multiple notification servers with horizontal scaling
- Introduce message queues to decouple components



Major Improvements in the Design

1. Move Database and Cache Out of Notification Server

In the initial design:

- Everything was handled inside one notification server

Problems:

- Tight coupling
- Difficult scaling
- Performance bottlenecks

Improvement:

- Database and cache are separated into independent components

Benefits:

- Independent scaling
- Better performance
- Easier maintenance

2. Horizontal Scaling of Notification Servers

Instead of one notification server:

- Multiple notification servers are added

Automatic horizontal scaling allows:

- Adding more servers during peak traffic
- Removing extra servers during low traffic

Benefits:

- Better scalability
- High availability
- Fault tolerance

3. Introduce Message Queues

Message queues are added between components.

Purpose:

- Decouple services
- Handle traffic spikes
- Improve reliability

Benefits:

- Asynchronous processing
- Better throughput
- Reduced dependency between components

Improved System Components

The best way to understand the architecture is from left to right.

1. Service 1 to N

These are services that generate notification requests.

Examples:

- Billing service
- Shopping service
- Marketing service

- Cron jobs
- Monitoring systems

Responsibilities:

- Call APIs exposed by notification servers
- Trigger notification events

2. Notification Servers

Notification servers are the core entry point of the system.

Responsibilities of Notification Servers

1. Provide Notification APIs

Notification servers expose internal APIs for sending notifications.

These APIs are:

- Accessible internally
- Accessible only to verified clients

Purpose:

- Prevent spam requests

Example API

None

POST <https://api.example.com/v/sms/send>

Request Body

```
{
  "to": [
    {
      "user_id": 123456
    }
  ],
  "from": {
    "email": "from_address@example.com"
  },
  "subject": "Hello, World!",
  "content": [
    {
      "type": "text/plain",
      "value": "Hello, World!"
    }
  ]
}
```

Contains notification data such as:

- User ID
- email
- Message content
- Notification type

2. Validation

Notification servers perform basic validations.

Examples:

- Validate email format
- Validate phone numbers
- Validate device tokens

3. Fetch Metadata

Notification servers retrieve additional information from:

- Cache
- Database

Examples:

- User preferences
- Device tokens
- Notification settings
- Templates

4. Push Events to Message Queues

After processing:

- Notification events are pushed into message queues

Purpose:

- Parallel processing
- Asynchronous delivery

3. Cache

The cache stores frequently accessed data.

Examples:

- User information
- Device information
- Notification templates

Benefits:

- Reduced database load
- Faster access
- Lower latency

4. Database (DB)

The database stores persistent system data.

Examples:

- User data
- Notification settings
- Device tokens
- Notification history

5. Message Queues

Message queues are one of the most important additions in the improved architecture.

Purpose:

- Remove dependency between components
- Buffer large traffic spikes
- Enable asynchronous processing

Separate Queue Per Notification Type

Each notification type gets its own queue.

Examples:

- iOS Push Queue
- Android Push Queue
- SMS Queue
- Email Queue

Benefits

Fault Isolation

If one third-party service fails:

- Other notification types continue working

Example:

- SMS service outage does not affect emails

Traffic Buffering

Queues absorb sudden traffic spikes.

Better Scalability

Workers can scale independently for each queue.

6. Workers

Workers are background servers responsible for processing notification events.

Responsibilities:

- Pull events from queues
- Process notifications
- Send notifications to third-party services

Benefits

- Parallel processing
- Independent scaling
- Improved throughput

7. Third-Party Services

Third-party services are responsible for delivering notifications.

Examples:

- APNS for iOS
- FCM for Android
- Twilio for SMS
- SendGrid for Email

8. User Devices

Notifications are finally delivered to:

- iOS devices
- Android devices
- SMS clients
- Email inboxes

End-to-End Notification Flow

Step 1: Service Sends Request

A service sends a notification request using APIs provided by notification servers.

Examples:

- Payment reminder
- Delivery update
- Promotional offer

Step 2: Fetch Metadata

Notification servers fetch required data from:

- Cache
- Database

Examples:

- Device token
- User preferences
- Email address
- Templates

Step 3: Push Event to Queue

Notification events are added to the corresponding message queue.

Example:

- iOS push event → iOS Push Queue
- SMS event → SMS Queue

Step 4: Workers Consume Events

Workers continuously pull notification events from queues.

Step 5: Send to Third-Party Services

Workers send processed notifications to:

- APNS
- FCM

- SMS provider
- Email provider

Step 6: Notification Delivered

Third-party services deliver notifications to end-user devices.

Advantages of Improved Design

1. High Scalability

- Components scale independently
- Workers can scale separately

2. Better Reliability

- Queue buffering prevents data loss
- Failures are isolated

3. Fault Tolerance

- One notification type failure does not affect others

4. Better Performance

- Asynchronous processing
- Reduced server overload

5. Extensibility

New notification types can be added easily.

Example:

- WhatsApp notifications
- Slack notifications
- In-app notifications

6. Improved Maintainability

Each component has clear responsibilities.

This simplifies:

- Debugging
- Monitoring
- Deployment
- Scaling